BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Practical No: 8

Division: C Batch: 2

Roll No: 3090367 PRN No: 12415134

Name of Student: Surabhi Harish Patel

Subject Name: Deep Learning

Problem Statement:

Implement a pre-trained BERT model for sentiment analysis or text classification on a sample dataset.

Algorithm :

Theory

Sentiment analysis is a Natural Language Processing (NLP) task used to identify the emotional tone or opinion expressed in a piece of text. It is commonly used for analyzing:

- Movie reviews
- Product reviews
- Customer feedback
- Social media posts
- Survey responses

Traditional machine-learning approaches often depend on manually designed features such as word frequencies or TF-IDF. Modern NLP systems use **Transformer-based models**, which can understand contextual relationships between words.

BERT is one of the most widely used Transformer models for NLP classification tasks. Instead of training a language model completely from scratch, a pre-trained BERT model can be fine-tuned on a smaller labeled dataset for a specific task such as sentiment analysis.

Dataset Description

The **Rotten Tomatoes movie-review dataset** contains short movie-review sentences labeled according to their sentiment.

For this assignment, the dataset contains two classes:

Label	Sentiment
0	Negative
1	Positive

A subset of the dataset is used for training and testing to make the experiment computationally manageable.
The implementation uses:

- **2,000 training samples**
- **500 test samples**

Methodology

Rotten Tomatoes Dataset
↓
Select Training & Testing Samples
↓
Explore Dataset
↓
BERT Tokenizer
↓
Tokenization + Padding
↓
Pre-trained BERT
↓
Add Classification Layer
↓
Fine-Tuning
↓
Model Evaluation
↓
Accuracy / Precision / Recall / F1
↓
Confusion Matrix
↓
Test New Reviews
↓
Save Trained Model

BERT

BERT stands for **Bidirectional Encoder Representations from Transformers**.

BERT is a Transformer-based language model developed for understanding natural language. The important characteristic of BERT is that it processes text **bidirectionally**.

Main characteristics

- Transformer-based architecture
- Bidirectional contextual understanding
- Pre-trained on large amounts of text
- Can be fine-tuned for downstream NLP tasks
- `uncased` means that the model does not distinguish between uppercase and lowercase letters

Evaluation Metrics:

- **Accuracy** – percentage of correct predictions.
- **Precision** – correctness of positive predictions.
- **Recall** – ability to identify actual positive samples.
- **F1-score** – combination of precision and recall.
- **Confusion Matrix** – shows correct and incorrect classifications.

Code :

```
# ================================================================
# BERT FOR SENTIMENT ANALYSIS
# Dataset: Rotten Tomatoes Movie Reviews
# Model: bert-base-uncased
# ================================================================


# ================================================================
# 1. INSTALL REQUIRED LIBRARIES
# ================================================================

# Run this cell/command once if using Google Colab or Jupyter

!pip -q install -U transformers datasets accelerate scikit-learn
matplotlib seaborn


# ================================================================
# 2. IMPORT LIBRARIES
# ================================================================

import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
```

```
from datasets import load_dataset

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    Trainer,
    TrainingArguments
)

from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix
)


# ================================================================
# 3. SET RANDOM SEED AND DEVICE
# ================================================================

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# Check whether GPU is available
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("=" * 60)
print("DEVICE INFORMATION")
print("=" * 60)

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))


# ================================================================
```

```
# 4. LOAD ROTTEN TOMATOES DATASET
# ================================================================

print("\n" + "=" * 60)
print("LOADING DATASET")
print("=" * 60)

dataset = load_dataset(
    "cornell-movie-review-data/rotten_tomatoes"
)

print(dataset)


# ================================================================
# 5. CREATE SMALL SAMPLE
# ================================================================

# Using a smaller sample makes the assignment faster to execute.
# Increase these values if you have a GPU.

TRAIN_SIZE = 2000
TEST_SIZE = 500

# Shuffle the training data
train_ds = dataset["train"].shuffle(
    seed=SEED
).select(
    range(TRAIN_SIZE)
)

# Shuffle the test data
test_ds = dataset["test"].shuffle(
    seed=SEED
).select(
    range(TEST_SIZE)
)

print("\nTraining examples:", len(train_ds))
print("Testing examples :", len(test_ds))


# ================================================================
# 6. DISPLAY SAMPLE REVIEWS
# ================================================================
```

```
print("\n" + "=" * 60)
print("SAMPLE REVIEWS")
print("=" * 60)

for i in range(5):

    print("\nReview:")
    print(train_ds[i]["text"])

    print("Label:", train_ds[i]["label"])

    if train_ds[i]["label"] == 0:
        print("Sentiment: NEGATIVE")
    else:
        print("Sentiment: POSITIVE")


# ================================================================
# 7. CHECK CLASS DISTRIBUTION
# ================================================================

print("\n" + "=" * 60)
print("CLASS DISTRIBUTION")
print("=" * 60)

labels = train_ds["label"]

unique_labels, counts = np.unique(
    labels,
    return_counts=True
)

print("Negative reviews:", counts[0])
print("Positive reviews:", counts[1])

plt.figure(figsize=(7, 5))

plt.bar(
    ["Negative", "Positive"],
    counts
)

plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
```

```
plt.ylabel("Number of Reviews")

plt.show()


# ================================================================
# 8. LOAD PRE-TRAINED BERT TOKENIZER
# ================================================================

print("\n" + "=" * 60)
print("LOADING BERT TOKENIZER")
print("=" * 60)

MODEL_NAME = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(
    MODEL_NAME
)

print("Tokenizer loaded successfully.")


# ================================================================
# 9. TOKENIZE DATASET
# ================================================================

print("\n" + "=" * 60)
print("TOKENIZING DATASET")
print("=" * 60)


def tokenize_function(examples):

    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=256
    )


# Tokenize training data
tokenized_train = train_ds.map(
    tokenize_function,
    batched=True
)
```

```python
# Tokenize test data
tokenized_test = test_ds.map(
    tokenize_function,
    batched=True
)

print("Tokenization completed.")


# ================================================================
# 10. TEST TOKENIZER
# ================================================================

sample_text = "This movie was absolutely fantastic!"

print("\nOriginal Text:")
print(sample_text)

print("\nTokens:")
print(tokenizer.tokenize(sample_text))

print("\nToken IDs:")
print(tokenizer(sample_text)["input_ids"])


# ================================================================
# 11. DATA COLLATOR
# ================================================================

data_collator = DataCollatorWithPadding(
    tokenizer=tokenizer
)

print("\nData collator created successfully.")


# ================================================================
# 12. LOAD PRE-TRAINED BERT MODEL
# ================================================================

print("\n" + "=" * 60)
print("LOADING PRE-TRAINED BERT MODEL")
print("=" * 60)

id2label = {
```

```
    0: "NEGATIVE",
    1: "POSITIVE"
}

label2id = {
    "NEGATIVE": 0,
    "POSITIVE": 1
}

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2,
    id2label=id2label,
    label2id=label2id
)

print("BERT model loaded successfully.")

trainable_parameters = sum(
    p.numel()
    for p in model.parameters()
    if p.requires_grad
)

print(
    "Trainable parameters:",
    trainable_parameters
)


# ================================================================
# 13. DEFINE EVALUATION METRICS
# ================================================================

def compute_metrics(eval_pred):

    logits, labels = eval_pred

    # Convert logits into predicted class
    predictions = np.argmax(
        logits,
        axis=-1
    )

    # Calculate precision, recall and F1
```

```
    precision, recall, f1, _ = (
        precision_recall_fscore_support(
            labels,
            predictions,
            average="binary",
            zero_division=0
        )
    )

    # Calculate accuracy
    accuracy = accuracy_score(
        labels,
        predictions
    )

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }


# ============================================================
# 14. DEFINE TRAINING PARAMETERS
# ============================================================

print("\n" + "=" * 60)
print("SETTING TRAINING PARAMETERS")
print("=" * 60)

training_args = TrainingArguments(

    # Directory for results
    output_dir="./bert_rotten_tomatoes_results",

    # Number of epochs
    num_train_epochs=1,

    # Training batch size
    per_device_train_batch_size=8,

    # Evaluation batch size
    per_device_eval_batch_size=16,
```

```
    # Learning rate
    learning_rate=2e-5,

    # Weight decay
    weight_decay=0.01,

    # Evaluate after every epoch
    eval_strategy="epoch",

    # Save after every epoch
    save_strategy="epoch",

    # Load best model at the end
    load_best_model_at_end=True,

    # Select best model using F1
    metric_for_best_model="f1",

    greater_is_better=True,

    # Logging
    logging_steps=50,

    # Disable external logging
    report_to="none",

    # Use FP16 if GPU is available
    fp16=torch.cuda.is_available(),

    # Reproducibility
    seed=SEED
)


# ================================================================
# 15. CREATE TRAINER
# ================================================================

trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=tokenized_train,
```

```
    eval_dataset=tokenized_test,

    processing_class=tokenizer,

    data_collator=data_collator,

    compute_metrics=compute_metrics
)

print("Trainer created successfully.")


# ================================================================
# 16. TRAIN / FINE-TUNE BERT
# ================================================================

print("\n" + "=" * 60)
print("STARTING BERT TRAINING")
print("=" * 60)

trainer.train()


# ================================================================
# 17. EVALUATE MODEL
# ================================================================

print("\n" + "=" * 60)
print("MODEL EVALUATION")
print("=" * 60)

metrics = trainer.evaluate()

for key, value in metrics.items():

    if isinstance(value, float):

        print(
            f"{key}: {value:.4f}"
        )

    else:

        print(
```

```
            f"{key}: {value}"
        )


# ================================================================
# 18. GENERATE PREDICTIONS
# ================================================================

print("\n" + "=" * 60)
print("GENERATING PREDICTIONS")
print("=" * 60)

pred_output = trainer.predict(
    tokenized_test
)

predictions = np.argmax(
    pred_output.predictions,
    axis=-1
)

true_labels = pred_output.label_ids

print("Predictions generated successfully.")


# ================================================================
# 19. CLASSIFICATION REPORT
# ================================================================

print("\n" + "=" * 60)
print("CLASSIFICATION REPORT")
print("=" * 60)

report = classification_report(

    true_labels,

    predictions,

    target_names=[
        "NEGATIVE",
        "POSITIVE"
    ],
```

```
    digits=4
)

print(report)


# ================================================================
# 20. CONFUSION MATRIX
# ================================================================

print("\n" + "=" * 60)
print("CONFUSION MATRIX")
print("=" * 60)

cm = confusion_matrix(
    true_labels,
    predictions
)

print(cm)

plt.figure(figsize=(7, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[
        "NEGATIVE",
        "POSITIVE"
    ],
    yticklabels=[
        "NEGATIVE",
        "POSITIVE"
    ]
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title(
    "BERT Sentiment Classification - Confusion Matrix"
)

plt.show()
```

```python
# ================================================================
# 21. PREDICT SENTIMENT OF NEW SENTENCES
# ================================================================

print("\n" + "=" * 60)
print("TESTING ON NEW SENTENCES")
print("=" * 60)


def predict_sentiment(texts):

    # Put model into evaluation mode
    model.eval()

    # Tokenize input sentences
    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=256
    )

    # Move inputs to model device
    inputs = {
        key: value.to(model.device)
        for key, value in inputs.items()
    }

    # Disable gradient calculation
    with torch.no_grad():

        outputs = model(**inputs)

        # Convert logits into probabilities
        probabilities = torch.softmax(
            outputs.logits,
            dim=-1
        )

        # Select highest probability class
        predictions = torch.argmax(
            probabilities,
```

```
            dim=-1
        )

    results = []

    for text, pred, probs in zip(
        texts,
        predictions,
        probabilities
    ):

        results.append({

            "text": text,

            "sentiment": id2label[
                pred.item()
            ],

            "confidence": float(
                probs[pred].item()
            )
        })

    return results


# New sentences
sample_texts = [

    "This movie was absolutely fantastic. I loved every minute of it!",

    "The movie was boring and disappointing.",

    "The acting was excellent and the story was very entertaining.",

    "I did not enjoy this movie at all.",

    "A wonderful film with great performances.",

    "The plot was terrible and the movie was a complete waste of time."
]


# Generate predictions
```

```
results = predict_sentiment(
    sample_texts
)


# Display predictions
for result in results:

    print("\nText:")
    print(result["text"])

    print(
        "Sentiment:",
        result["sentiment"]
    )

    print(
        "Confidence:",
        f"{result['confidence']:.2%}"
    )


# ================================================================
# 22. SAVE FINE-TUNED MODEL
# ================================================================

print("\n" + "=" * 60)
print("SAVING MODEL")
print("=" * 60)

SAVE_PATH = "./bert_rotten_tomatoes_model"

# Save trained model
trainer.save_model(
    SAVE_PATH
)

# Save tokenizer
tokenizer.save_pretrained(
    SAVE_PATH
)

print(
    "Model saved successfully at:",
    SAVE_PATH
```

```
)


# ====================================================================
# 23. LOAD SAVED MODEL
# ====================================================================

print("\n" + "=" * 60)
print("LOADING SAVED MODEL")
print("=" * 60)

loaded_tokenizer = AutoTokenizer.from_pretrained(
    SAVE_PATH
)

loaded_model = AutoModelForSequenceClassification.from_pretrained(
    SAVE_PATH
)

print("Saved model loaded successfully.")


# ====================================================================
# 24. FINAL SUMMARY
# ====================================================================

print("\n" + "=" * 60)
print("ASSIGNMENT COMPLETED")
print("=" * 60)

print("Dataset : Rotten Tomatoes")
print("Model   : BERT (bert-base-uncased)")
print("Task    : Binary Sentiment Classification")
print("Classes : NEGATIVE / POSITIVE")

print("\nEvaluation Metrics:")

if "eval_accuracy" in metrics:
    print(
        "Accuracy :",
        f"{metrics['eval_accuracy']:.4f}"
    )

if "eval_precision" in metrics:
    print(
```

```
        "Precision:",
        f"{metrics['eval_precision']:.4f}"
    )

if "eval_recall" in metrics:
    print(
        "Recall   :",
        f"{metrics['eval_recall']:.4f}"
    )

if "eval_f1" in metrics:
    print(
        "F1 Score :",
        f"{metrics['eval_f1']:.4f}"
    )

print("\nBERT sentiment analysis completed successfully.")
```

Output :

LOADING DATASET

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 8530
    })
    validation: Dataset({
        features: ['text', 'label'],
        num_rows: 1066
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 1066
    })
})
```

Training examples: 2000
Testing examples : 500

SAMPLE REVIEWS

Review:
. . . plays like somebody spliced random moments of a chris rock routine into what is otherwise a cliche-riddled but self-serious spy thriller .

Label: 0
Sentiment: NEGATIVE

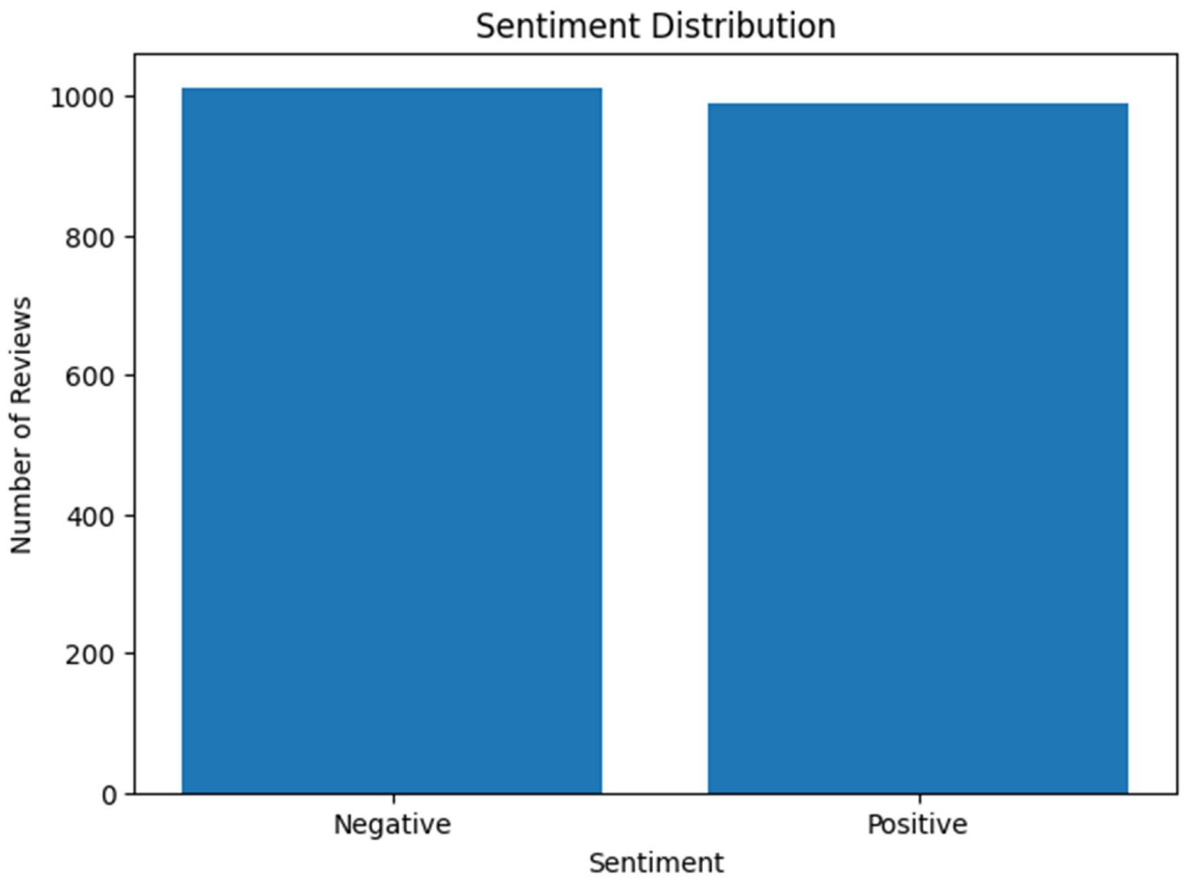
Review:
michael moore has perfected the art of highly entertaining , self-aggrandizing , politically motivated documentary-making , and he's got as potent a topic as ever here .
Label: 1
Sentiment: POSITIVE

Review:
. . . too gory to be a comedy and too silly to be an effective horror film .
Label: 0
Sentiment: NEGATIVE

Review:
a graceful , contemplative film that gradually and artfully draws us into a world where the personal and the political get fatally intertwined .
Label: 1
Sentiment: POSITIVE

Review:
the fact that the 'best part' of the movie comes from a 60-second homage to one of demme's good films doesn't bode well for the rest of it .
Label: 0
Sentiment: NEGATIVE

CLASS DISTRIBUTION
Negative reviews: 1011
Positive reviews: 989



TOKENIZING DATASET
Tokenization completed.

Original Text:
This movie was absolutely fantastic!

Tokens:
['this', 'movie', 'was', 'absolutely', 'fantastic', '!']

Token IDs:
[101, 2023, 3185, 2001, 7078, 10392, 999, 102]

Data collator created successfully.

LOADING PRE-TRAINED BERT MODEL
Loading weights: 100%
199/199 [00:00<00:00, 3424.43it/s]
[transformers] **BertForSequenceClassification LOAD REPORT** from: bert-base-uncased
Key | Status |
----------+------------+-

cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
BERT model loaded successfully.
Trainable parameters: 109483778

STARTING BERT TRAINING
[250/250 18:37, Epoch 1/1]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.395823	0.414912	0.828000	0.818565	0.818565	0.818565

Writing model shards: 0%| | 0/1 [00:00<?, ?it/s]
[transformers] There were missing keys in the checkpoint model loaded:
['bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias',
'bert.encoder.layer.0.attention.output.LayerNorm.weight',
'bert.encoder.layer.0.attention.output.LayerNorm.bias',
'bert.encoder.layer.0.output.LayerNorm.weight', 'bert.encoder.layer.0.output.LayerNorm.bias',
'bert.encoder.layer.1.attention.output.LayerNorm.weight',
'bert.encoder.layer.1.attention.output.LayerNorm.bias',
'bert.encoder.layer.1.output.LayerNorm.weight', 'bert.encoder.layer.1.output.LayerNorm.bias',
'bert.encoder.layer.2.attention.output.LayerNorm.weight',
'bert.encoder.layer.2.attention.output.LayerNorm.bias',
'bert.encoder.layer.2.output.LayerNorm.weight', 'bert.encoder.layer.2.output.LayerNorm.bias',
'bert.encoder.layer.3.attention.output.LayerNorm.weight',
'bert.encoder.layer.3.attention.output.LayerNorm.bias',
'bert.encoder.layer.3.output.LayerNorm.weight', 'bert.encoder.layer.3.output.LayerNorm.bias',
'bert.encoder.layer.4.attention.output.LayerNorm.weight',
'bert.encoder.layer.4.attention.output.LayerNorm.bias',
'bert.encoder.layer.4.output.LayerNorm.weight', 'bert.encoder.layer.4.output.LayerNorm.bias',
'bert.encoder.layer.5.attention.output.LayerNorm.weight',
'bert.encoder.layer.5.attention.output.LayerNorm.bias',
'bert.encoder.layer.5.output.LayerNorm.weight', 'bert.encoder.layer.5.output.LayerNorm.bias',

'bert.encoder.layer.6.attention.output.LayerNorm.weight',
'bert.encoder.layer.6.attention.output.LayerNorm.bias',
'bert.encoder.layer.6.output.LayerNorm.weight', 'bert.encoder.layer.6.output.LayerNorm.bias',
'bert.encoder.layer.7.attention.output.LayerNorm.weight',
'bert.encoder.layer.7.attention.output.LayerNorm.bias',
'bert.encoder.layer.7.output.LayerNorm.weight', 'bert.encoder.layer.7.output.LayerNorm.bias',
'bert.encoder.layer.8.attention.output.LayerNorm.weight',
'bert.encoder.layer.8.attention.output.LayerNorm.bias',
'bert.encoder.layer.8.output.LayerNorm.weight', 'bert.encoder.layer.8.output.LayerNorm.bias',
'bert.encoder.layer.9.attention.output.LayerNorm.weight',
'bert.encoder.layer.9.attention.output.LayerNorm.bias',
'bert.encoder.layer.9.output.LayerNorm.weight', 'bert.encoder.layer.9.output.LayerNorm.bias',
'bert.encoder.layer.10.attention.output.LayerNorm.weight',
'bert.encoder.layer.10.attention.output.LayerNorm.bias',
'bert.encoder.layer.10.output.LayerNorm.weight', 'bert.encoder.layer.10.output.LayerNorm.bias',
'bert.encoder.layer.11.attention.output.LayerNorm.weight',
'bert.encoder.layer.11.attention.output.LayerNorm.bias',
'bert.encoder.layer.11.output.LayerNorm.weight',
'bert.encoder.layer.11.output.LayerNorm.bias'].
[transformers] There were unexpected keys in the checkpoint model loaded:
['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.gamma',
'bert.encoder.layer.0.attention.output.LayerNorm.beta',
'bert.encoder.layer.0.attention.output.LayerNorm.gamma',
'bert.encoder.layer.0.output.LayerNorm.beta', 'bert.encoder.layer.0.output.LayerNorm.gamma',
'bert.encoder.layer.1.attention.output.LayerNorm.beta',
'bert.encoder.layer.1.attention.output.LayerNorm.gamma',
'bert.encoder.layer.1.output.LayerNorm.beta', 'bert.encoder.layer.1.output.LayerNorm.gamma',
'bert.encoder.layer.2.attention.output.LayerNorm.beta',
'bert.encoder.layer.2.attention.output.LayerNorm.gamma',
'bert.encoder.layer.2.output.LayerNorm.beta', 'bert.encoder.layer.2.output.LayerNorm.gamma',
'bert.encoder.layer.3.attention.output.LayerNorm.beta',
'bert.encoder.layer.3.attention.output.LayerNorm.gamma',
'bert.encoder.layer.3.output.LayerNorm.beta', 'bert.encoder.layer.3.output.LayerNorm.gamma',
'bert.encoder.layer.4.attention.output.LayerNorm.beta',
'bert.encoder.layer.4.attention.output.LayerNorm.gamma',
'bert.encoder.layer.4.output.LayerNorm.beta', 'bert.encoder.layer.4.output.LayerNorm.gamma',
'bert.encoder.layer.5.attention.output.LayerNorm.beta',
'bert.encoder.layer.5.attention.output.LayerNorm.gamma',
'bert.encoder.layer.5.output.LayerNorm.beta', 'bert.encoder.layer.5.output.LayerNorm.gamma',
'bert.encoder.layer.6.attention.output.LayerNorm.beta',
'bert.encoder.layer.6.attention.output.LayerNorm.gamma',
'bert.encoder.layer.6.output.LayerNorm.beta', 'bert.encoder.layer.6.output.LayerNorm.gamma',
'bert.encoder.layer.7.attention.output.LayerNorm.beta',
'bert.encoder.layer.7.attention.output.LayerNorm.gamma',

'bert.encoder.layer.7.output.LayerNorm.beta', 'bert.encoder.layer.7.output.LayerNorm.gamma',
'bert.encoder.layer.8.attention.output.LayerNorm.beta',
'bert.encoder.layer.8.attention.output.LayerNorm.gamma',
'bert.encoder.layer.8.output.LayerNorm.beta', 'bert.encoder.layer.8.output.LayerNorm.gamma',
'bert.encoder.layer.9.attention.output.LayerNorm.beta',
'bert.encoder.layer.9.attention.output.LayerNorm.gamma',
'bert.encoder.layer.9.output.LayerNorm.beta', 'bert.encoder.layer.9.output.LayerNorm.gamma',
'bert.encoder.layer.10.attention.output.LayerNorm.beta',
'bert.encoder.layer.10.attention.output.LayerNorm.gamma',
'bert.encoder.layer.10.output.LayerNorm.beta',
'bert.encoder.layer.10.output.LayerNorm.gamma',
'bert.encoder.layer.11.attention.output.LayerNorm.beta',
'bert.encoder.layer.11.attention.output.LayerNorm.gamma',
'bert.encoder.layer.11.output.LayerNorm.beta',
'bert.encoder.layer.11.output.LayerNorm.gamma'].
[250/250 19:07, Epoch 1/1]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.395823	0.414912	0.828000	0.818565	0.818565	0.818565

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning:
'pin_memory' argument is set as true but no accelerator is found, then device pinned memory
won't be used.
  super().__init__(loader)

MODEL EVALUATION
[32/32 01:15]

Training Loss	Validation Loss	Epoch	Accuracy	Precision	Recall	F1
0.395823	0.414912	1	0.828000	0.818565	0.818565	0.818565

eval_loss: 0.4149
eval_accuracy: 0.8280
eval_precision: 0.8186
eval_recall: 0.8186
eval_f1: 0.8186

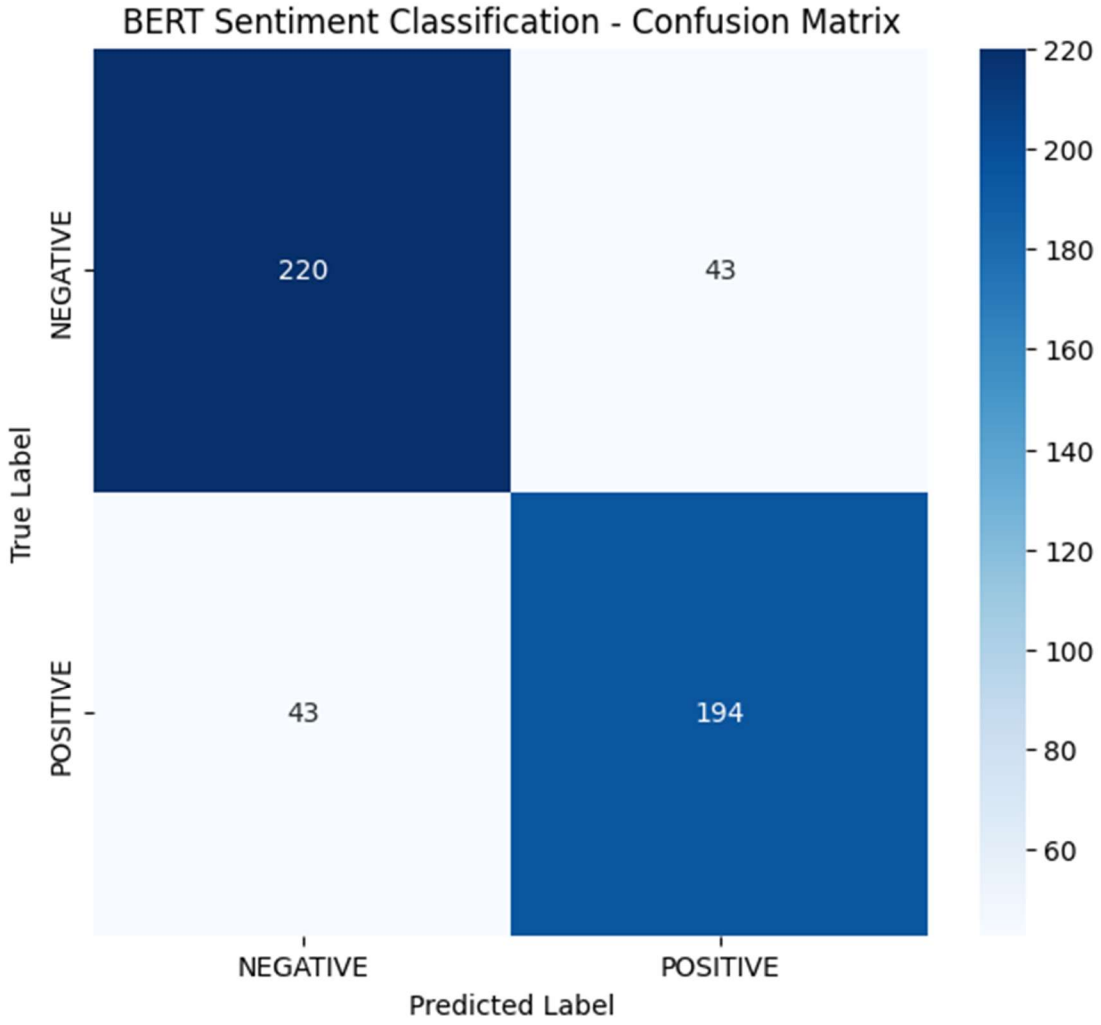
==
GENERATING PREDICTIONS
==
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning:
'pin_memory' argument is set as true but no accelerator is found, then device pinned memory
won't be used.
  super().__init__(loader)
Predictions generated successfully.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
NEGATIVE	0.8365	0.8365	0.8365	263
POSITIVE	0.8186	0.8186	0.8186	237
accuracy			0.8280	500
macro avg	0.8275	0.8275	0.8275	500
weighted avg	0.8280	0.8280	0.8280	500

CONFUSION MATRIX

[[220 43]
[43 194]]



==========

TESTING ON NEW SENTENCES

==========

Text:
This movie was absolutely fantastic. I loved every minute of it!
Sentiment: POSITIVE
Confidence: 94.18%

Text:
The movie was boring and disappointing.
Sentiment: NEGATIVE
Confidence: 91.78%

Text:
The acting was excellent and the story was very entertaining.
Sentiment: POSITIVE
Confidence: 94.53%

Text:
I did not enjoy this movie at all.
Sentiment: NEGATIVE
Confidence: 89.55%

Text:
A wonderful film with great performances.
Sentiment: POSITIVE
Confidence: 94.80%

Text:
The plot was terrible and the movie was a complete waste of time.
Sentiment: NEGATIVE
Confidence: 89.80%

SAVING MODEL
Writing model shards: 100%
1/1 [00:11<00:00, 11.70s/it]
Model saved successfully at: ./bert_rotten_tomatoes_model

LOADING SAVED MODEL
Loading weights: 100%
201/201 [00:00<00:00, 3211.27it/s]
Saved model loaded successfully.

ASSIGNMENT COMPLETED
Dataset : Rotten Tomatoes
Model : BERT (bert-base-uncased)

Task   : Binary Sentiment Classification
Classes : NEGATIVE / POSITIVE

Evaluation Metrics:
Accuracy : 0.8280
Precision: 0.8186
Recall   : 0.8186
F1 Score : 0.8186

BERT sentiment analysis completed successfully.